

Adatstruktúrák és algoritmusok

Miért lényeges az adatok tárolási formája?
Idő- és tárhelykapacitás-bonyolultság

Bevezetés

- 1 A tárgyról: a három szint
- 2 Motiváló példa: a kölcsönök kiegyenlítése
- 3 Alapfogalmak, definíciók
- 4 Időbonyolultság
- 5 Tárhatalom-bonyolultság
- 6 Aszimptotikus jelölések
- 7 Vissza a példához

- 1 A tárgyról: a három szint
- 2 Motiváló példa: a kölcsönök kiegyenlítése
- 3 Alapfogalmak, definíciók
- 4 Időbonyolultság
- 5 Tárhatalom-bonyolultság
- 6 Aszimptotikus jelölések
- 7 Vissza a példához

Egy informatikai rendszer három szintje

Egy alkalmazási rendszer kifejlesztése során három fő szintet szokás megkülönböztetni. Példa: helyjegyfoglalási rendszer egy InterCity vonatjáratra.

A szint neve	A szint jellemzése	A szint fogalomrendszere
Felső szint	alkalmazói szint, modellalkotás	útvonalak, szerelvények, dátumok, helyfoglalások
Középső szint	modellezési szint, algoritmizálás	file-ok, táblázatok, listák, adatrekordok, stringek, fák
Alsó szint	csupasz gépszint	objektumok, műveletek, gépi reprezentálásuk, bitek, byte-ok

A középső szint az, amivel ez a tárgy foglalkozik.

A felső szint — az alkalmazó területe

- ▶ Ő tudja, hogy milyen útvonalakon, mely napokon közlekedtet szerelvényeket, és milyen ezen szerelvények összetétele a helyfoglalás szempontjából.
- ▶ A helyfoglalási rendszer **kereteit** neki kell kijelölnie: mi a fontos a rendszerében, mi az el nem hagyható tény, és mi az, ami elhanyagolható.
- ▶ Tudjon-e majd a rendszer olyan igényt is kielégíteni, hogy *az utas ablaknál akar ülni, de csak a menetiránnyal azonos irányban, ne háttal?*

Miért kritikus ez a szint?

Az alkalmazó a valóság egy modelljéhez a kereteket alkotja meg. A későbbi üzemelő rendszer **paramétereit, képességeit, rugalmasságát és használhatóságát** ez a szint döntően meghatározza.

Középső szint — a modell gyakorlati megvalósítása

- ▶ Magába foglalja az egyes adatkezelési, számítási, tárolási **módszereket**.
- ▶ Itt tisztázódik a file-ok rendszere; rögzítik a használandó táblázatokat, listákat és azok szerkezetét, az adatrekordok felépítését.
- ▶ Kialakításra kerülnek a keresési módszerek és az adatmódosítások módszerei — miután eldöntötték, **mit és hogyan tárolnak**.

Alsó szint — a gépek, berendezések szintje

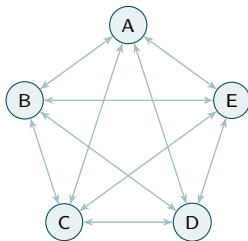
- ▶ Itt nincs modell, nincs szerelvény, dátum, útvonal.
- ▶ Az adatok, a tárolási szerkezetek és a rajtuk végzett műveletsorok **bitek és byte-ok** özőneként és átalakításaiként jelennek meg.
- ▶ Fontos az egymástól térben és időben is távoli eszközök közötti **kommunikáció** lehetősége és milyensége.

- 1 A tárgyról: a három szint
- 2 Motiváló példa: a kölcsönök kiegyenlítése**
- 3 Alapfogalmak, definíciók
- 4 Időbonyolultság
- 5 Tárhatalmítás-bonyolultság
- 6 Aszimptotikus jelölések
- 7 Vissza a példához

A feladat

Adott egy n fős társaság. Az egyes tagok időnként pénzt kérnek kölcsön egymástól. Mindenki felírja, hogy **kitől mennyit kért** kölcsön, és **kinek mennyit adott** kölcsön.

A társaság tagjai időről időre összejönnek az összegyűlt tartozásokat **kiegyenlíteni**.



Mindenki megkeres mindenkit $\implies$ „némi kavarodás”, és nem kis időbe telik.

1. megoldás: a tartozás-mátrix

Gyűjtsük össze egy táblázatba a *tartozik–követel* összesítéseket. A táblázat **sora** mutatja, hogy ki tartozik, az **oszlopa**, hogy kinek tartozik.

	Aladár	Béla	Cecil	Dávid	Ede
Aladár	—	5	3	1	2
Béla	1	—	2	3	4
Cecil	5	1	—	1	2
Dávid	2	3	4	—	6
Ede	5	5	1	4	—

Például: Dávid tartozik Ede-nek 6-tal, Cecil tartozik Aladárnak 5-tel.

1. megoldás: mennyibe kerül?

Erőforrásigény n fő esetén

- ▶ **Tárigény:** $n \cdot n - n = n(n - 1)$ rekesz (a főátló üres).
- ▶ **Találkozók száma:** $\frac{n(n - 1)}{2}$, ahol a két fél kölcsönösen kiegyenlíti az egymással szembeni adósságát.

Mind a két formulában szerepel egy **négyzetes tag**!

Mit jelent ez?

Ha a társaság mérete mondjuk 10-szeresre nő, akkor a vele kapcsolatos szervezési és tárolási munka egyaránt körülbelül **100-szorosra** emelkedik.

Nem éppen biztató következtetés. . . **Van azonban jobb megoldás is!**

2. megoldás: elég egyetlen szám

Nem kell minden alkalommal mindenkinek feljegyezni, hogy kitől mennyit kért vagy kinek mennyit adott. **Elegendő, ha mindenki csak egyetlen számot tárol** és azt módosítja kölcsönzés esetén: ez pedig az éppen aktuális **össztartozása** a többiek felé.

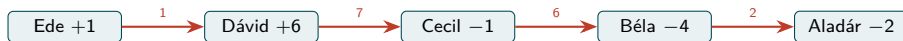
A fenti táblázatból ez így kapható meg: képezzük a sorösszegeket és az oszlopösszegeket, majd mindenkinél vonjuk ki a tartozásból (sorösszeg) a követel (oszlopösszeg) értéket.

	Aladár	Béla	Cecil	Dávid	Ede	Sorösszeg	Össztartozás
Aladár	—	5	3	1	2	11	$11 - 13 = -2$
Béla	1	—	2	3	4	10	$10 - 14 = -4$
Cecil	5	1	—	1	2	9	$9 - 10 = -1$
Dávid	2	3	4	—	6	15	$15 - 9 = 6$
Ede	5	5	1	4	—	15	$15 - 14 = 1$
Oszlopösszeg	13	14	10	9	14		

Tárolni **csak az össztartozás oszlopot** kell, ami n szám. A kigyűjtés sem szükséges.

A kiegyenlítés: láncolatos elszámolás

Össztartozások: $A = -2$, $B = -4$, $C = -1$, $D = +6$, $E = +1$ (az összegük mindig 0).



- ▶ Aladárnak tartoznak 2-vel: ezt **Béla** megadja Aladárnak.
- ▶ Most már Bélának tartoznak 6-tal: azt **Cecil** adja meg Bélának.
- ▶ Cecilnek tartoznak 7-tel: amit megad neki **Dávid**.
- ▶ Így Dávidnak lesz 1 hiánya, amit pedig **Ede** éppen meg tud adni.

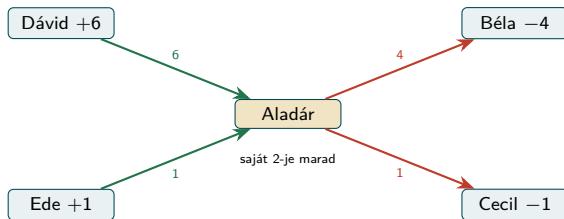
Eredmény

Ha n fő van, akkor a szükséges találkozók száma legfeljebb $n - 1$.

Alternatíva: központi elszámoló

A láncolatos megoldásnál nem árt, ha Béla és Cecil rendelkeznek némi **plusz pénzzel**, hogy fizetni tudjanak az elején. Ez elkerülhető:

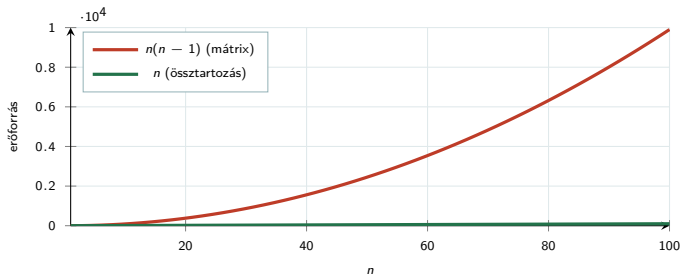
- ▶ Akik tartoznak (**Dávid** és **Ede**), azok mindegyike mondjuk **Aladárnak** adja oda a tartozását.
- ▶ Ebből a pénzből Aladár egyenlíti ki a hiányt azoknál, akik pénzre várnak (Aladár, Béla, Cecil).



Ez is $n - 1 = 4$ átutalás, viszont **senkinek nem kell megelőlegeznie** semmit.

A két megoldás összevetése

n	1. megoldás (mátrix)		2. megoldás (össztartozás)	
	tárrekesz $n(n-1)$	találkozó $\frac{n(n-1)}{2}$	tárrekesz n	találkozó $n-1$
5	20	10	5	4
10	90	45	10	9
100	9 900	4 950	100	99
1 000	999 000	499 500	1 000	999
10^6	$\approx 10^{12}$	$\approx 5 \cdot 10^{11}$	10^6	$\approx 10^6$



Ha a méretek 10-szeresre nőnek, akkor a 2. megoldásnál a tárolás és a szervezési munka is csak körülbelül **10-szeresre** fog nőni (szemben az 1. megoldás 100-szorosával).

Érdemes tehát azon elgondolkodni...

- ▶ milyen **adatokat** tárolunk,
- ▶ milyen **formában** tároljuk őket,
- ▶ azokon milyen **műveleteket** végzünk,

hogy a kívánt eredményre jussunk, és az a lehető legkisebb erőforrás-lekötéssel és energiafelhasználással valósuljon meg.

Fontos: **ugyanaz az információ** van tárolva mindkét megoldásban — csak más formában!

A jó adatszerkezet nem gyorsítja fel a rossz algoritmust: *feleslegessé teszi*.

Mit veszítünk cserébe?

A második megoldás nem „ingyen” jobb — **kevesebb információt** tárol:

Kérdés	mátrix	össztartozás
Mennyi az össztartozásom?	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Mennyivel tartozom <i>Bélának</i> ?	$\mathcal{O}(1)$	nem válaszolható meg elveszett
Ki kinek tartozik?	megvan	
Egy új kölcsön rögzítése	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Tárigény	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$

Az adatszerkezet-tervezés lényege

Előre el kell dönteni, **milyen kérdéseket** akarunk feltenni. Ami a feladat szempontjából elhanyagolható, azt nem kell tárolni — és éppen ez hozza a nyereséget.

Ahhoz, hogy mindezt **pontosan** meg tudjuk fogalmazni, kellenek a bonyolultsági fogalmak.

- 1 A tárgyról: a három szint
- 2 Motiváló példa: a kölcsönök kiegyenlítése
- 3 Alapfogalmak, definíciók**
- 4 Időbonyolultság
- 5 Tárhatalom-bonyolultság
- 6 Aszimptotikus jelölések
- 7 Vissza a példához

Algoritmus

Véges sok, egyértelműen végrehajtható elemi lépés sorozata, amely tetszőleges megengedett bemenetből véges idő alatt előállítja a kívánt kimenetet.

Az input mérete

Az **n inputméret** az a mérőszám, amelyben a bemenet nagyságát mérjük. Feladatfüggő, hogy mi:

- ▶ társaság létszáma, tömb elemszáma, lista hossza: n ;
- ▶ gráfnál: a csúcsok és élek száma (n és m);
- ▶ egy N egész szám feldolgozásánál gyakran a **leírási hossza**, azaz $n = \lfloor \log_2 N \rfloor + 1$ bit.

A bonyolultságot mindig **az inputméret függvényében** adjuk meg, hiszen azt kérdezzük: *hogyan skálázódik a módszer, ha a feladat nő?*

A számítási modell: elemi lépések

A méréshez rögzíteni kell, mi számít **egy lépésnek**. A szokásos RAM-modellben egységnyi költségű:

- ▶ egy értékadás, egy aritmetikai művelet ($+$, $-$, $\cdot$, $/$),
- ▶ egy összehasonlítás, egy logikai művelet,
- ▶ egy tömbelem elérése az indexe alapján, egy ugrás.

Miért jó ez így?

- ▶ Nem függ a konkrét géptől, programnyelvtől, fordítótól.
- ▶ Egy kétszer gyorsabb gép a lépésszámot nem változtatja meg, csak a konstans szorzót — ezért a **nagyságrendet** vizsgáljuk.

Sokszor elég egyetlen **domináns műveletet** számolni (pl. rendezésnél az összehasonlítást, a példánkban a találkozókat).

- 1 A tárgyról: a három szint
- 2 Motiváló példa: a kölcsönök kiegyenlítése
- 3 Alapfogalmak, definíciók
- 4 Időbonyolultság**
- 5 Tárhatalom-bonyolultság
- 6 Aszimptotikus jelölések
- 7 Vissza a példához

Az időbonyolultság definíciója

Legyen $\mathcal{A}$ egy algoritmus, és jelölje I_n az n méretű megengedett bemenetek halmazát. Egy konkrét $I \in I_n$ bemeneten legyen $t_{\mathcal{A}}(I)$ a végrehajtott elemi lépések száma.

Definíció: időbonyolultság

$$T_{\mathcal{A}}(n) = \max_{I \in I_n} t_{\mathcal{A}}(I) \quad \text{legrosszabb eset (worst case),}$$

$$A_{\mathcal{A}}(n) = \sum_{I \in I_n} p(I) \cdot t_{\mathcal{A}}(I) \quad \text{átlagos eset (average case), } p \text{ eloszlás mellett,}$$

$$B_{\mathcal{A}}(n) = \min_{I \in I_n} t_{\mathcal{A}}(I) \quad \text{legjobb eset (best case).}$$

- ▶ Alapértelmezésben a **legrosszabb esetet** vizsgáljuk: ez *garancia*, minden bemenetre érvényes felső korlát.
- ▶ Az átlagos eset kiszámításához ismerni kell a bemenetek **valószínűség-eloszlását** — ez sokszor vitatható feltevés.

Példa: lineáris keresés

Keressük az x értéket az $A[1 : n]$ tömbben, előlről hátrafelé haladva.

eset	összehasonlítás	mikor?
legjobb	$B(n) = 1$	$x = A[1]$
legrosszabb	$T(n) = n$	$x = A[n]$ vagy $x \notin A$
átlagos (benne van, egyenletesen)	$A(n) = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2}$	—

- 1 A tárgyról: a három szint
- 2 Motiváló példa: a kölcsönök kiegyenlítése
- 3 Alapfogalmak, definíciók
- 4 Időbonyolultság
- 5 Tárhatalom-bonyolultság**
- 6 Aszimptotikus jelölések
- 7 Vissza a példához

A tárbonyolultság definíciója

Jelölje $s_{\mathcal{A}}(I)$ az $\mathcal{A}$ algoritmus által az I bemeneten **egyidejűleg** lefoglalt tárrekeszek maximális számát.

Definíció: tárkapacitás-bonyolultság

$$S_{\mathcal{A}}(n) = \max_{I \in I_n} s_{\mathcal{A}}(I)$$

az $\mathcal{A}$ algoritmus **tárbonyolultsága** (legrosszabb esetben), az n inputméret függvényében.

- ▶ **Teljes tárigény:** a bemenet tárolása is beleszámít.
- ▶ **Munkatár (segédvár):** csak a bemeneten felül használt hely. Ezt szokás vizsgálni, ha azt kérdezzük, mennyi *plusz* hely kell.
- ▶ **Helyben (in situ) dolgozik** az algoritmus, ha a munkatára $\mathcal{O}(1)$, azaz konstans.
- ▶ Rekurziónál a **veremmélység** is tárigény: $\mathcal{O}(\log n)$ vagy $\mathcal{O}(n)$ is lehet, még ha nem foglalunk le tömböt.

Idő–tár átváltás (time–space tradeoff)

Gyakran a memória feláldozásával idő nyerhető, és fordítva.

Megoldás	idő	tár
érték újraszámolása minden kérdésnél előre kiszámolt táblázat (gyorsítótár)	nagy kicsi	kicsi nagy
kölcsönök: tartozás-mátrix	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
kölcsönök: össztartozás	$\mathcal{O}(n)$	$\mathcal{O}(n)$

A bevezető példa azért tanulságos, mert ott **nem** kellett átváltani: a jobb adatábrázolás egyszerre csökkentette az időt és a tárat. Ilyen akkor lehetséges, ha az eredeti reprezentáció **redundáns** volt a feladathoz képest.

- 1 A tárgyról: a három szint
- 2 Motiváló példa: a kölcsönök kiegyenlítése
- 3 Alapfogalmak, definíciók
- 4 Időbonyolultság
- 5 Tárhatalom-bonyolultság
- 6 Aszimptotikus jelölések**
- 7 Vissza a példához

Miért aszimptotikus becslés?

- ▶ A pontos lépésszám függ a géptől, a fordítótól, a megvalósítástól — ezek **konstans szorzót** jelentenek.
- ▶ Kis n -re minden módszer gyors; a különbség **nagy n -re** mutatkozik meg.
- ▶ Ezért a *növekedési ütemet* vizsgáljuk, elhagyva a konstansokat és az alacsonyabb rendű tagokat:

$$3n^2 + 17n + 2000 \rightsquigarrow n^2.$$

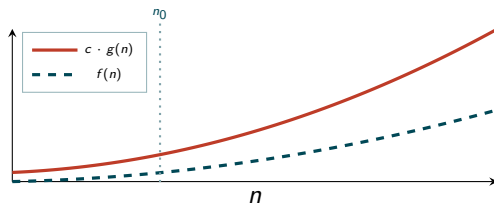
A vizsgált függvények

A továbbiakban $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ pozitív értékű függvények (lépésszám, tárigény).

Definíció ($\mathcal{O}$)

$$f(n) = \mathcal{O}(g(n)) \iff \exists c > 0, \exists n_0 \in \mathbb{N} : \forall n \geq n_0 : f(n) \leq c \cdot g(n).$$

Szavakban: f **legfeljebb olyan gyorsan nő**, mint g (konstans szorzó erejéig, elég nagy n -ekre).



Példák: $3n^2 + 17n + 2000 = \mathcal{O}(n^2)$, $n = \mathcal{O}(n^2)$, $\log_2 n = \mathcal{O}(n)$, $n(n-1)/2 = \mathcal{O}(n^2)$.

- ① **Konstansok elhagyása:** $c \cdot f(n) = \mathcal{O}(f(n))$ minden $c > 0$ -ra.
- ② **Összegekszabály:** $\mathcal{O}(f) + \mathcal{O}(g) = \mathcal{O}(\max\{f, g\})$ — egymás utáni programrészeknél a lassabbik dominál.
- ③ **Szoratszabály:** $\mathcal{O}(f) \cdot \mathcal{O}(g) = \mathcal{O}(f \cdot g)$ — egymásba ágyazott ciklusoknál.
- ④ **Tranzitivitás:** ha $f = \mathcal{O}(g)$ és $g = \mathcal{O}(h)$, akkor $f = \mathcal{O}(h)$.
- ⑤ **Polinomok:** ha f egy k -adfokú polinom pozitív főegyütthatóval, akkor $f(n) = \mathcal{O}(n^k)$.
- ⑥ **Logaritmus alapja mindegy:** $\log_a n = \mathcal{O}(\log_b n)$, hiszen $\log_a n = \frac{\log_b n}{\log_b a}$.
- ⑦ **Rendezés:** minden $\varepsilon > 0$ esetén

$$1 \prec \log n \prec n^\varepsilon \prec 2^n \prec n! \prec n^n,$$

ahol $\prec$ azt jelenti, hogy a bal oldalán lévő függvény és a jobb oldalán lévő függvény aránya $n \rightarrow \infty$ esetben a 0-hoz tart..

Nagyságrendek a gyakorlatban

Tegyük fel, hogy egy elemi lépés 1 nanoszekundum (10^{-9} s).

$T(n)$	$n = 100$	$n = 10^4$	$n = 10^6$	$n = 10^9$
$\log_2 n$	≈ 7 ns	13 ns	20 ns	30 ns
n	0,1 μ s	10 μ s	1 ms	1 s
$n \log_2 n$	0,7 μ s	0,13 ms	20 ms	30 s
n^2	10 μ s	0,1 s	17 perc	32 év
n^3	1 ms	17 perc	32 év	—
2^n	$\approx 10^{13}$ év	—	—	—

Következtetés

A **nagyságrend** sokkal többet számít, mint a gép sebessége: egy $1000\times$ gyorsabb géppel a 2^n -es algoritmus is csak kb. 10-zel nagyobb n -t bír el, míg a lineáris algoritmus $1000\times$ nagyobb feladatot old meg.

A bevezető példa formálisan

	1. megoldás (mátrix)	2. megoldás (össztartozás)
tárolt adat	$n(n-1)$ szám	n szám
tárbonyolultság $S(n)$	$n^2 - n = \mathcal{O}(n^2)$	$n = \mathcal{O}(n)$
kiegyenlítés (találkozók)	$\frac{n(n-1)}{2} = \mathcal{O}(n^2)$	$n - 1 = \mathcal{O}(n)$
egy kölcsön rögzítése	$\mathcal{O}(1)$	$\mathcal{O}(1)$
az össztartozások előállítása	$\mathcal{O}(n^2)$	nem kell (folyamatos)

A „10-szeres” megfigyelés formálisan

Ha $T(n) = \mathcal{O}(n^2)$, akkor $T(10n) \approx 100 T(n)$; ha $T(n) = \mathcal{O}(n)$, akkor $T(10n) \approx 10 T(n)$.

Miért volt megkerülhető a négyzetes tag?

- ▶ A mátrix **minden párra** külön tárolta a viszonyt: $\binom{n}{2}$ adat. Ez a feladathoz képest **redundáns** volt.
- ▶ A kiegyenlítéshez viszont csak az **egyenleg** kell: minden személyhez egyetlen szám, mert az összegek *összevonhatók* (a tartozása b felé és b tartozása a felé kiüti egymást).
- ▶ Az össztartozások összege mindig 0, ezért mindig van olyan sorrend, amelyben egyetlen „pénzáram” végigfut a résztvevőkön: innen az $n - 1$ átutalás.

Az általános tanulság

Először azt kell tisztázni, **milyen kérdésekre** kell válaszolni, és ehhez **milyen adatot** kell megőrizni. A jó adatábrázolás nagyságrendet csökkent — nem konstans szorzót.

- ▶ A rendszerfejlesztés három szintje közül a **középső** (modellezés, algoritmizálás) a tárgyunk témája: itt dől el, mit és hogyan tárolunk.
- ▶ A kölcsön-példa mutatja: ugyanazon feladat két adatábrázolása között **nagyságrendi** különbség lehet ($\mathcal{O}(n^2)$ vs. $\mathcal{O}(n)$) — egyszerre időben és tárban.
- ▶ **Időbonyolultság:** $T(n) = \max_{I \in I_n} t(I)$, az elemi lépések száma az inputméret függvényében (legrosszabb, átlagos, legjobb eset).
- ▶ **Tárbonyolultság:** $S(n) = \max_{I \in I_n} s(I)$, az egyidejűleg lefoglalt tárrekeszek maximális száma (teljes tár vs. munkatár; in situ = $\mathcal{O}(1)$ munkatár).
- ▶ Nagy n -re a **nagyságrend** dönt, nem a gép sebessége.

- [1] *Adatstruktúrák, algoritmusok* — egyetemi jegyzet, 1. fejezet: Bevezetés (a tárgyról; alapvető fogalmak, definíciók).
- [2] Cormen, Leiserson, Rivest, Stein: *Új algoritmusok* (Introduction to Algorithms). Sclor Kiadó.
- [3] Rónyai Lajos, Ivanyos Gábor, Szabó Réka: *Algoritmusok*. Typotex, Budapest.
- [4] D. E. Knuth: *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*. Addison-Wesley.